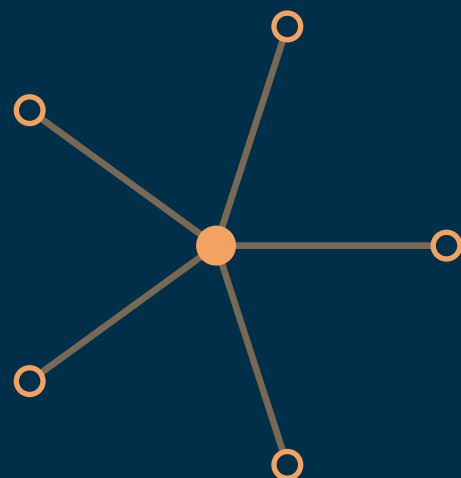




Training Data Reconstruction

from Molecular Classifiers

*If a model memorized its training data —
can you make it confess?*

AiChemist Team · Copenhagen Hackathon 2026

- Membership Inference
- ECFP4 Fingerprints
- Gradient Norm

The Challenge

Two trained neural networks classify molecules by biological activity. The training data is hidden.

Goal: submit 1000 molecules per task that exactly match the original training set.

Task 1 — Binary Classifier

- Architecture: 2048 → 512 → 128 → 2
- No regularisation (no dropout, no batchnorm)
- Train accuracy: ~100% | Test accuracy: ~65%
- Deliberately overfit — memorised its training data

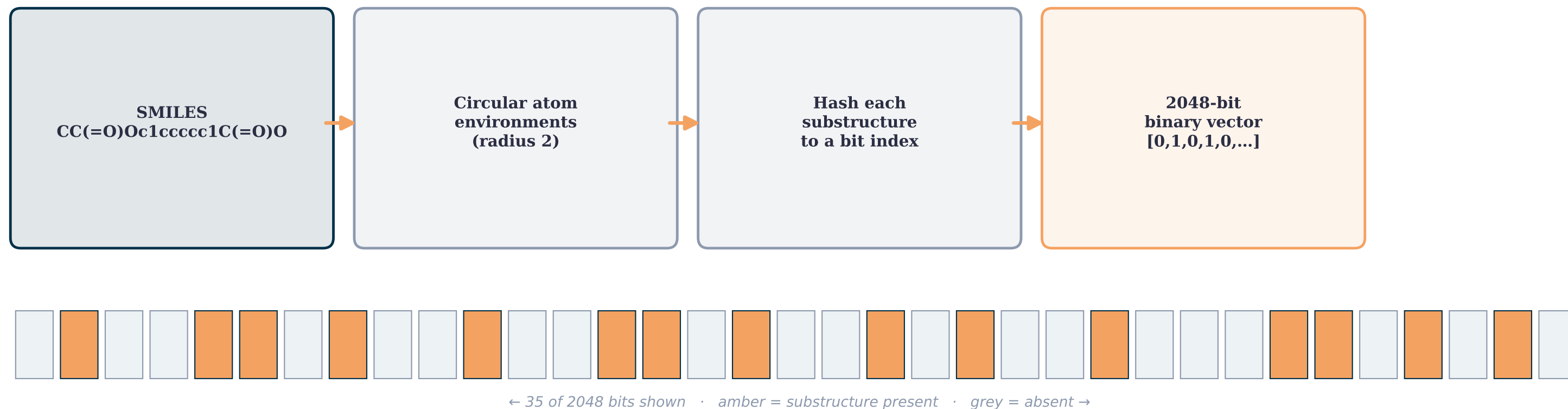
Task 2 — 4-Class Classifier

- Architecture: 2048 → [BN + ReLU + Dropout] → 512 → 256 → 4
- AdamW optimiser + early stopping at epoch 4
- Train accuracy: ~90% | Test accuracy: ~86%
- Well regularised — generalises to new molecules

Both models take an ECFP4 fingerprint as input (Morgan algorithm · radius 2 · 2048 bits)

ECFP4 Fingerprints

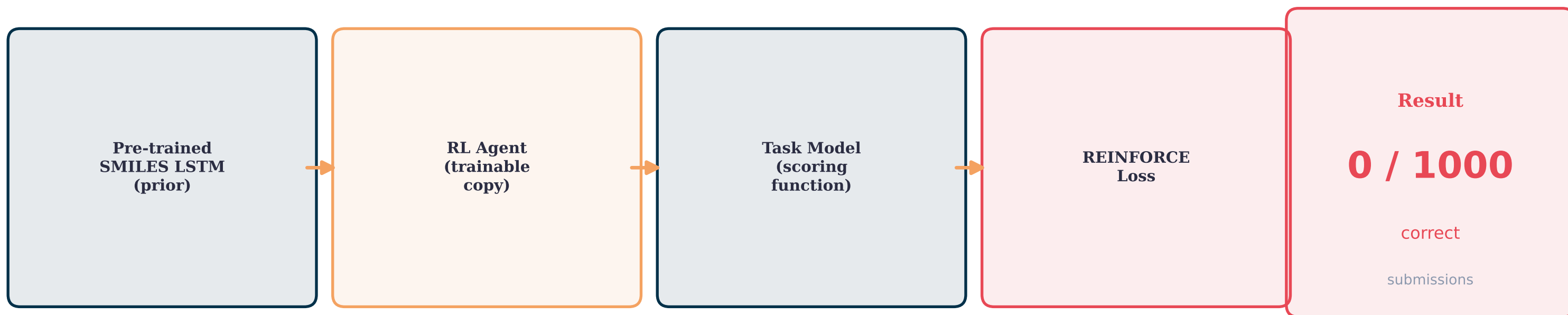
How a molecule becomes a row of 0s and 1s



- ▶ Same molecule always gives the same fingerprint
- ▶ Similar molecules share many bits (measured by Tanimoto similarity)
- ▶ The mapping is one-way — you cannot reconstruct a molecule from bits alone

Approach 1 — Generative Pipeline

REINVENT RL · attempted first



```
augmented_ll = prior_log_prob(smiles) +  $\sigma$  × confidence(smiles)

loss         = mean( ( augmented_ll - agent_log_prob(smiles) )2 )

# agent steers toward molecules the task model scores highly
```

Generates plausible drug-like molecules — but not the specific molecules in the training set.

The Key Insight — Raw Logits

Softmax hides what logits reveal

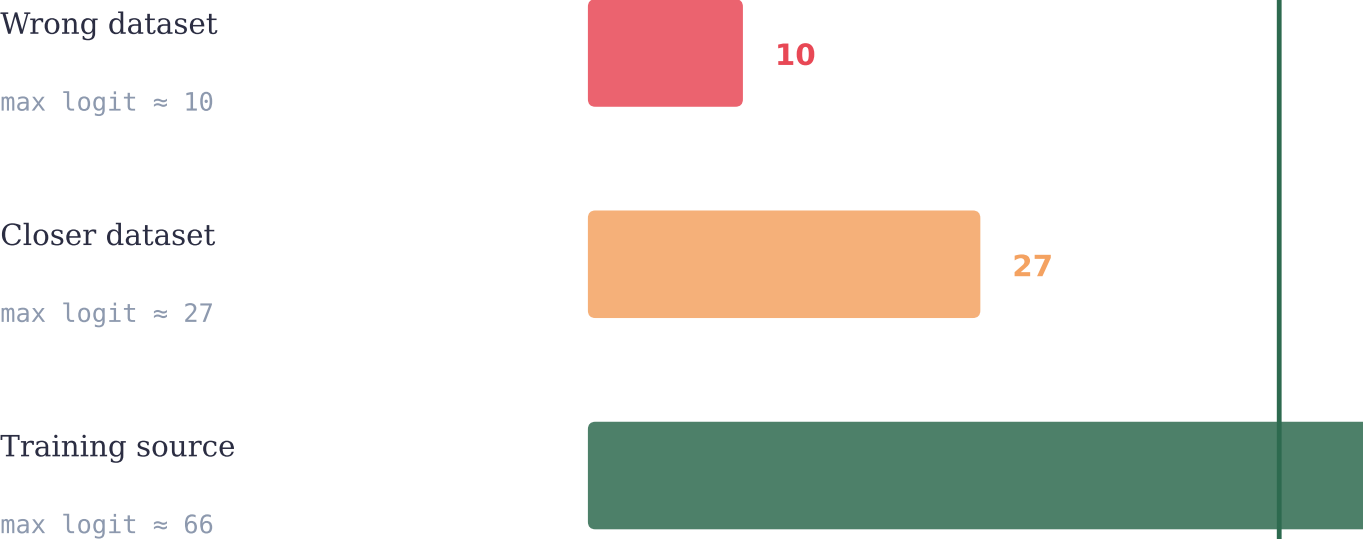
Softmax confidence — saturates

logit = 10	→	p = 0.9999	≈ 1.00
logit = 27	→	p = 0.9999	≈ 1.00
logit = 66	→	p = 1.0000	≈ 1.00

All look identical.

Cannot distinguish training from non-training molecules.

Raw logit — full dynamic range



6× logit gap reveals the training source.

The model's internal certainty betray which molecules it memorised.

Submission 1 — At a Glance

Six steps · fold-based · gradient norm

01

Load Dataset

CARBIDE data_splits.tsv

1988 molecules total

Step 1

02

Filter Folds

Keep Fold != 0

1588 training molecules

Step 2

03

Fingerprint

ECFP4 · 2048 bits

RDKit Morgan radius 2

Step 3

04

Run Model

Forward pass

Get logit scores

Step 4

05

Gradient Norm

Backprop loss

Small norm = memorised

Step 5

06

Export Top-1000

Task 1: top-1000

Task 2: 250 per class

Step 6

Steps 1 & 2 — Load and Filter

Finding the training molecules

Step 1 — Load Dataset

CARBIDE's data_splits.tsv contains every molecule used in the train/test cross-validation splits.

SMILES	Cluster_ID	Fold
<chem>CC1=Nc2cc(C(F)...</chem>	26	0
<chem>C#CC(0)(C=CC1)...</chem>	232	1
<chem>C#CC1(0)CCC2C...</chem>	1	1
<chem>Brc1c(NC2=NCCN...</chem>	14	3

Total: 1988 molecules across 5 folds (Fold 0–4)

Step 2 — Filter to Training Folds

With seed = 0, fold 0 was held out as the test set. We keep only folds 1, 2, 3, 4 — the training set.

Fold 0 (test)	Fold 1	Fold 2	Fold 3	Fold 4
	~397	~397	~397	~397
X excluded	kept (training)			

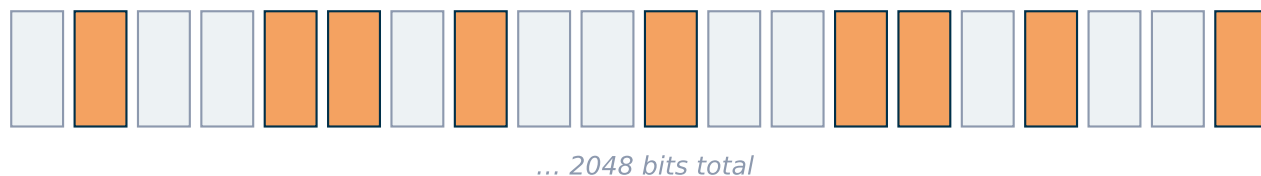
```
splits = pd.read_csv('data_splits.tsv', sep='\t')  
  
train = splits[splits['Fold'] != 0]['SMILES']  
  
# → 1588 training molecules
```

Steps 3 & 4 — Fingerprint & Forward Pass

Converting molecules to numbers, then asking the model

Step 3 — ECFP4 Fingerprint

Each molecule is converted to a 2048-bit binary vector.
This is the language the model understands.



```
from rdkit.Chem import rdFingerprintGenerator

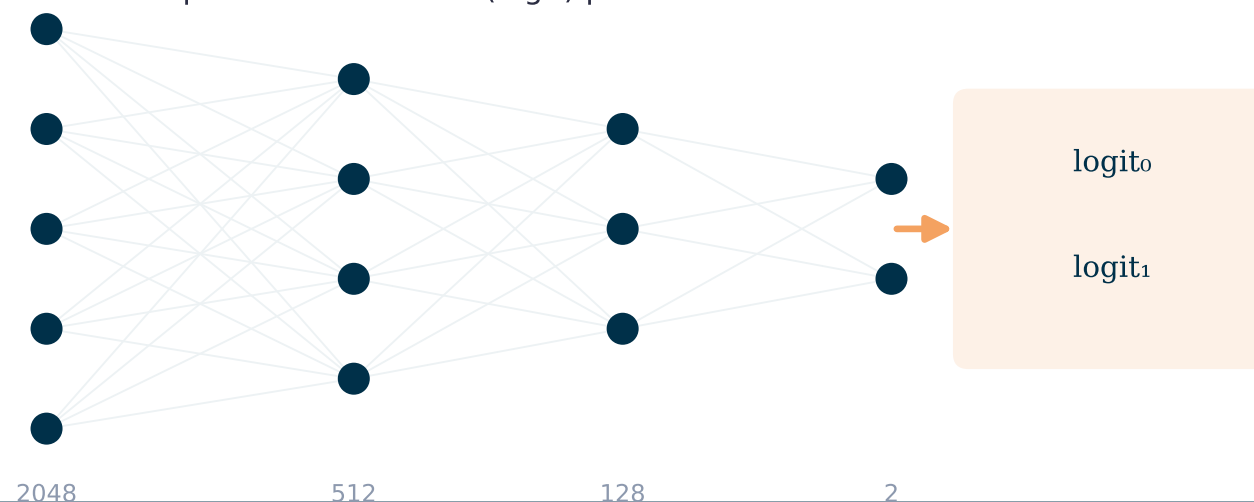
gen = GetMorganGenerator(radius=2, fpSize=2048)

fp = gen.GetFingerprintAsNumPy(mol)

# → array([0,1,0,0,1,1,...], dtype=float32)
```

Step 4 — Run the Model

Feed each fingerprint into the task model.
The model outputs one raw score (logit) per class.



```
X = torch.from_numpy(np.stack(fingerprints))

logits = model(X) # shape: (n_molecules, n_classes)

# → tensor([[ -3.1,  8.7], [-12.4, 22.1], ...])
```


Step 5 — Gradient Norm

The membership signal: how much does the model need to change its mind?

Analogy

Think of a student taking an exam.

A student who memorised the answer → no hesitation → gradient ≈ 0

A student who is guessing → unsure → needs to adjust → gradient > 0

The Mathematics

We backpropagate the loss through the model to the input:

```
x.requires_grad_(True)

F.cross_entropy(model(x), pred).backward()
```

Then measure how large that gradient is:

```
gradient_norm = x.grad.norm()
```

What the signal looks like

Training molecule
(memorised)

0.002

Training molecule
(memorised)

0.009

Non-training
(unknown)

0.180

Non-training
(unknown)

0.310

← smaller gradient norm

= model is already at a loss minimum

= this molecule was memorised

Step 6 — Rank and Export

Sort by gradient norm, take the top molecules

Sort all 1588 training-fold molecules by gradient norm (ascending).
Smallest norm first = most memorised = most likely in training set.

Task 1 — Binary Classifier

Take the 1000 molecules with the smallest gradient norm.

```
norms = gradient_norms(model1, fingerprints)

top = np.argsort(norms)[:1000]

result = [smiles[i] for i in top]
```

norm range in top-1000: 0.0000 – 0.0363

Output

task1_submission.csv

1000 molecules

Task 2 — 4-Class Classifier

For each of the 4 classes, compute the gradient norm using that class as the target. Assign each molecule to its most 'natural' class, take top 250 per class.

```
for c in range(4):

    norms[:,c] = gradient_norms(model2, fps, class_idx=c)

assigned = np.argmin(norms, axis=1)
```

Output

task2_submission.csv

250 × 4 classes = 1000 molecules

Results — The Journey

Random
Generation

Large Database
Screening

Logit Signal
Discovery

Gradient
Norm



0 correct

Prior generates valid
molecules, not specific ones

0 correct

Wrong dataset —
logits all similar (~10)

60+ correct

Right domain found
via logit magnitude jump

Best result

Principled approach:
fold-based + $\partial L / \partial x \approx 0$

Key finding

Raw logit magnitude — not softmax confidence — is the discriminative membership signal. A 6× logit gap between the wrong and right dataset is unmistakable.

Takeaways

- 1 Dataset identification matters more than algorithm sophistication
- 2 Use raw logits — not softmax — for membership inference in overfit models
- 3 Gradient norm is the principled signal: $\partial L / \partial x \approx 0$ at training data points
- 4 A generative model cannot reconstruct specific training molecules
- 5 Cross-validation fold structure reveals the train/test split boundary